



รายงาน

เรื่อง Socket Programming

ระบบบริหารจัดการคิว (Queue Management System)

จัดทำโดย

นายสิริราช ชูทรัพย์ รหัสนักศึกษา 6710451429

เสนอ

ผศ.ดร. สุขุมล กิตติสิน

รายงานนี้เป็นส่วนหนึ่งของการศึกษาค้นคว้า

รายวิชา 01418351 หลักการสื่อสารคอมพิวเตอร์และการประมวลผลบนคลาวด์

ภาคเรียนที่ 1 ปีการศึกษา 2569

1. ภาพรวมและวัตถุประสงค์ของโปรแกรม

โปรแกรมนี้เป็น Network Application สำหรับการบริหารจัดการคิวร้านอาหาร โดยพัฒนาระบบภายใต้สถาปัตยกรรม Client-Server จุดประสงค์หลักคือเพื่อแก้ปัญหาการจัดการคิวในโลกจริง ที่จำเป็นต้องมีการจัดสรรคิวให้สอดคล้องกับ "ขนาดของโต๊ะ" (Party Size)

ระบบถูกออกแบบให้รองรับการทำงาน 2 บทบาท ได้แก่ ลูกค้า (Customer) ที่สามารถกรับคิวและตรวจสอบเวลารอ และ พนักงาน (Counter Staff) ที่สามารถเรียกคิวข้ามลำดับให้ตรงกับขนาดโต๊ะที่เพิ่งว่างลงได้ โดยเซิร์ฟเวอร์จะทำหน้าที่จัดเก็บสถานะคิวทั้งหมดแบบแยกหมวดหมู่และแยกขนาดอย่างเป็นระบบ

2. สถาปัตยกรรมเครือข่ายและการใช้ Transport Layer

โปรแกรมนี้ทำงานผ่าน TCP (Transmission Control Protocol) โดยมีสถาปัตยกรรมดังนี้:

- **Reliable & In-order Delivery:** ระบบคิวเป็นระบบที่ “ข้อมูลลัดคิวกันไม่ได้” และ “ข้อมูลหายไม่ได้” การใช้ TCP ช่วยรับประกันว่าข้อมูลการจองคิวจะส่งถึงเซิร์ฟเวอร์อย่างครบถ้วนและเรียงตามลำดับเวลาจริง (Chronological order)
- **Persistent Connection:** ไคลเอนต์และเซิร์ฟเวอร์จะสร้างการเชื่อมต่อเพียงครั้งเดียวและคงสถานะไว้ (Keep-alive) เพื่อส่งคำสั่งตอบโต้กันผ่านลูป (Loop) ทำให้ประหยัด Overhead ในการทำ 3-way Handshake ใหม่ทุกครั้งที่ใช้กดเมนู
- **High Concurrency Handling:** เซิร์ฟเวอร์ใช้ ExecutorService (Thread Pool) ในการบริหารจัดการทรัพยากร ทำให้สามารถรองรับการเชื่อมต่อจากไคลเอนต์หลายเครื่องพร้อมกันได้อย่างเสถียร

3. การออกแบบ Application-Layer Protocol: SQP

เพื่อให้ไคลเอนต์และเซิร์ฟเวอร์สื่อสารกันได้อย่างเข้าใจ จึงได้ออกแบบโปรโตคอลใหม่ชื่อ SQP (Smart Queue Protocol) ซึ่งเป็น Text-based Protocol ที่อ่านและประมวลผลได้ง่าย

3.1. กฎของไวยากรณ์ (Message Syntax)

- ใช้เครื่องหมาย | ในการแบ่งฟิลด์ข้อมูล (Header)
- ใช้เครื่องหมาย : ในการจับคู่ตัวแปรและค่าของมัน (Key-Value Pair)

3.2. โครงสร้าง Request Message (Client -> Server)

รูปแบบ: **CMD:** <คำสั่ง> | **TYPE:** <ประเภทคิว/ผู้ใช้> | **SIZE:** <ขนาดโตะ> | **PASS:** <รหัสผ่าน> ตัวอย่างการใช้งาน:

- ลูกค้าขอคิว 4 คน: **CMD:GET_Q|TYPE:NORMAL|SIZE:2-4persons**
- พนักงานเรียกคิว 4 คน: **CMD:NEXT_Q|TYPE:NORMAL|SIZE:2-4 persons**
- ขอประเมินเวลารอ: **CMD:ESTIMATE_TIME|TYPE:ALL**

3.3. โครงสร้าง Response Message (Server -> Client)

Status Code	Status Phrase	ความหมาย
200	OK	การทำงานสำเร็จตามที่ร้องขอ เช่น กดเรียกคิวสำเร็จ
201	Created	ระบบทำการบันทึกการจองคิวใหม่สำเร็จ และ ออกหมายเลขคิวให้เรียบร้อยแล้ว
400	Bad Request	คำสั่งไม่ถูกต้อง ไวยากรณ์ ข้อความผิดพลาด (Syntax Error) ข้อมูลไม่ครบถ้วน หรือส่งคำสั่งที่เซิร์ฟเวอร์ไม่รู้จัก
401	Unauthorized	ปฏิเสธการทำงานเนื่องจากสิทธิ์ไม่เพียงพอ เช่น ใส่รหัสผ่านผิด
404	Not found	ไม่พบข้อมูลที่ดำเนินการ เช่น พนักงานกดเรียกคิวในหมวดหมู่ที่ไม่มีลูกค้าย่อยอยู่เลย

รูปแบบ: **STATUS:**<รหัสสถานะ>|**Q_NO:**<หมายเลขคิว>|**MSG:**<ข้อความแจ้งเตือน> ตัวอย่างการใช้งาน:

- จองคิวสำเร็จ: STATUS:201|Q_NO:NB-001|MSG:Queue Created Successfully (2-4 persons)
- พนักงานเรียกคิว: STATUS:200|Q_NO:NB-001|MSG:Calling Normal Queue (2-4 persons)
- รหัสผ่านพนักงานผิด: STATUS:401|Q_NO:-|MSG:Unauthorized - Invalid Password

4. อัลกอริทึมและจุดเด่นของการพัฒนา (Key Highlights)

- **Complex Data Structure:** เซิร์ฟเวอร์ไม่ได้ใช้แค่ตัวแปรนับตัวเลขธรรมดา แต่ใช้โครงสร้างข้อมูลแบบ Queue<Integer> (Linked List) ถึง 6 ชุด เพื่อแยกเก็บคิวตามประเภท (VIP/Normal) และขนาดโต๊ะ (1, 2-4, 6-8) อย่างชัดเจน
- **Application-Layer Security:** มีการออกแบบระบบ Authentication ในตัวโปรโตคอล โดยบังคับให้มีการส่ง CMD:AUTH เพื่อล็อกอินก่อน และทุกคำสั่งที่เปลี่ยนแปลงสถานะของคิว (เช่น NEXT_Q) จะต้องแนบ Parameter PASS เสมอ เพื่อป้องกันการสวมรอยของข้อมูล
- **Real-time Queue Estimation:** ระบบสามารถดึงขนาดของ Queue ที่เหลืออยู่จริงในเซิร์ฟเวอร์มาคำนวณและตอบกลับเป็นระยะเวลารอ (Estimated Time) ได้ทันทีแบบเรียลไทม์